



— TECHNICAL ARCHITECTURE & ENGINEERING SPECIFICATION

Cross-Platform Implementation Guide

System architecture, network protocols, audio/video pipelines, spatial hit-testing, and adaptation blueprints for Snap Spectacles, Meta Quest (Horizon OS), and Android XR.

AUTHOR

Krazyy Krunal
All Things Krazyy

SPECIFICATION TARGET

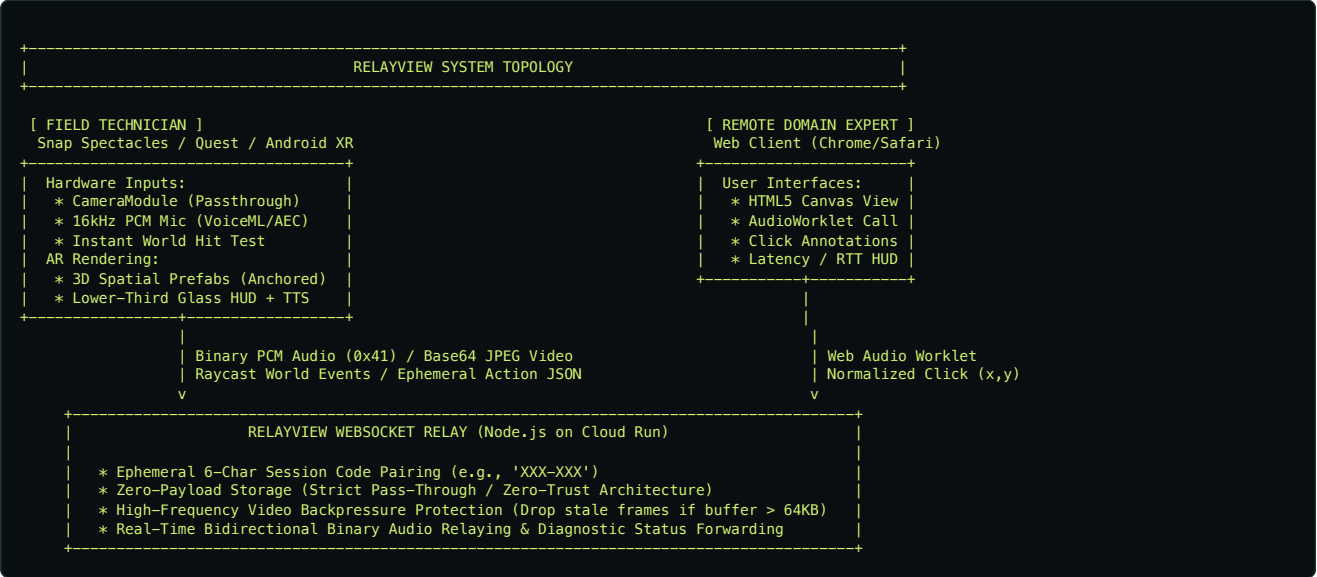
Spectacles · Quest · Android XR
SnapOS · Horizon OS · OpenXR

VERSION & DATE

v1.0 Production Architecture
October 2026

RelayView (originally incubated as *Remote ARsistance*) is a real-time, hands-free spatial collaboration platform. It bridges on-site technicians operating in demanding physical environments (telecom, avionics, industrial manufacturing, precision data centers) with off-site domain experts connected via standard web browsers.

While conventional remote assistance forces users to hold smartphones or tablets—neutralizing their manual dexterity—RelayView leverages wearable augmented reality to project synchronized two-way voice, head-mounted POV video, and **3D spatial annotations anchored directly onto physical equipment** in real time.



Core Architectural Tenets

- **Zero-Friction Authentication:** Technicians cannot type complex passwords while wearing smart glasses in the field. Pairing is handled via ephemeral 6-character Crockford Base32 keys generated on device boot.
- **Lightweight Binary Protocol:** Rather than forcing heavy, battery-draining WebRTC peer negotiation on low-power XR SoCs, RelayView employs a clean binary WebSocket architecture optimized for Qualcomm Snapdragon XR2.
- **Decoupled Spatial State:** 2D screen interactions from the web portal are translated through real-time raycasting into persistent 6DoF spatial entities anchored in the physical world.
- **Fail-Safe Hardware Lifecycle:** Audio drivers and camera providers are kept persistent during call toggles to eliminate hardware lockups on embedded XR platforms.

2.1 Protocol Framing Specification

Communication between the XR client, relay server, and web portal is divided into **control JSON messages** and **high-bandwidth binary audio/video payloads** over a single secure WebSocket (`wss://`):

Payload Type	Encoding	Header / Framing	Description & Semantic Meaning
Binary PCM Audio	Raw PCM 16-bit LE	0x41 (1 Byte Tag)	Bidirectional voice call chunks. Followed by interleaved 16-bit little-endian samples at 16,000 Hz. Packet length is odd ($2N + 1$ bytes).
Video Frame	Base64 JPEG	FRAME_END suffix	Camera snapshot transmitted from XR headset. Server inspects suffix and relays directly to web peer without JSON decode overhead.
Session Handshake	JSON text	{"action":"create-session"}	XR client initiates. Server responds with {"status":"created","sessionCode":"ABC-DEF"}. Web client joins via {"action":"join-session"}.
Spatial Annotation	JSON text	{"action":"annotate"}	Expert sends normalized viewport coordinates {"data":{"x":0.48,"y":0.35,"label":"Torque Bolt"}}.
Mic Diagnostics	JSON text	{"action":"mic-status"}	Triggered by the headset watchdog when audio input stalls. Forwarded to web portal to guide device reboot or mic permission check.

2.2 Video Pipeline & Dynamic Quality Profiling

The video streamer in `CameraStreamer.js` operates on a cooperative render-target loop, balancing bandwidth against compression latency:

```
// CameraStreamer.js - Dynamic Quality Engine
const PROFILES = {
  crisp: { scale: 0.85, quality: 72, fps: 20 }, // High detail for circuit boards
  balanced: { scale: 0.65, quality: 60, fps: 15 }, // Standard field diagnostic
  fast: { scale: 0.50, quality: 48, fps: 12 } // Low-bandwidth cellular failover
};

function adaptQualityByLatency(rttMs) {
  if (rttMs < 90) setQualityProfile("crisp");
  else if (rttMs < 220) setQualityProfile("balanced");
  else setQualityProfile("fast");
}
```

2.3 Audio Subsystem & Hardware Protection (XR2)

On Qualcomm Snapdragon XR2 platforms (including Snap Spectacles Gen 5 and Meta Quest), calling `microphone.stop()` or re-instantiating the audio driver while the camera pipeline is active can cause the hardware DSP to stall, terminating the camera feed.

⚠ Critical Hardware Finding (XR2 Engine Lifecycle)

Never de-allocate or stop native `MicrophoneAudioProvider` or `AudioComponent` instances during call termination. Instead, transition a software state flag (`calling = false`) to discard outgoing buffers while preserving hardware driver warm-state.

```
// VoiceCall.js - Audio Capture & Mic Stall Watchdog
updateEvent.bind(function () {
  if (!calling || !isJoined()) return;

  const frameSize = microphone.maxFrameSize || 16384;
  const audioFrame = new Float32Array(frameSize);
  let shape = microphone.getAudioFrame(audioFrame);
  let count = (shape && isFinite(shape.x)) ? (shape.x | 0) : 0;

  if (count > 0 && socket.readyState === 1) {
    zeroTicks = 0;
    const sendCount = Math.min(count, 2048);
    const packet = new Uint8Array(sendCount * 2 + 1);
    packet[0] = 0x41; // 'A' packet tag
```

```

    for (let i = 0; i < sendCount; i++) {
      const s = Math.max(-1.0, Math.min(1.0, audioFrame[i]));
      const val = s < 0 ? Math.round(s * 32768) : Math.round(s * 32767);
      packet[1 + i * 2] = val & 255;
      packet[2 + i * 2] = (val >> 8) & 255;
    }
    socket.send(packet);
  } else {
    zeroTicks++;
    // 300 ticks @ 60 FPS = 5.0 seconds of mic silence during call
    if (!stallPrompted && zeroTicks >= 300 && sentAudioPackets === 0) {
      stallPrompted = true;
      ChatUI.showAlert("⚠ Mic silent · Check Spectacles\nRestart device if issue persists", 7.0);
      socket.send(JSON.stringify({ action: "mic-status", status: "waiting" }));
    }
  }
});

```

3.1 Screen-to-World Raycasting Math

When a remote expert clicks at coordinate (x, y) in [0, 1] on the live video canvas, RelayView computes a 3D ray from the headset's focal center into the reconstructed scene:

```
// 2D Normalized Viewport -> 3D Spatial Entity
function renderAnnotation(label, xNorm, yNorm) {
  // 1. Invert Y coordinate from HTML canvas standard to XR viewport standard
  const screenPos = new vec2(xNorm, 1.0 - yNorm);

  // 2. Perform Instant World Hit Test against real-world depth mesh
  const hitTestResult = instantWorldHitTest.hitTest(screenPos);
  let worldPosition;

  if (hitTestResult && hitTestResult.isValid()) {
    worldPosition = hitTestResult.position; // Surface hit point
  } else {
    // Fallback: Project ray to standard task distance (1.2 meters)
    const cameraPos = perspectiveCam.getSceneObject().getTransform().getWorldPosition();
    const cameraForward = perspectiveCam.getSceneObject().getTransform().forward;
    worldPosition = cameraPos.add(cameraForward.uniformScale(1.2));
  }

  // 3. Instantiate Billboard Anchor facing user
  const pin = createSpatialTagPrefab(label, worldPosition);
  pin.getComponent("Component.LookAtComponent").target = perspectiveCam.getSceneObject();
}
```

3.2 Design System & Ergonomic Specification

RelayView's design language is built specifically for high-stress industrial environments with variable ambient lighting.

Obsidian Base #0b1014	Electric Lime #d4f36f	Slate Border #1f2c2e	Sage Secondary #88a994

- **Lower-Third HUD Placement:** AR notifications must not occlude the center of the technician's field of view. `ScreenTransform` anchors are set to `[-0.85, 0.85, -0.92, -0.62]`, positioning information in the comfortable downward gaze arc.
- **High-Contrast Double Borders:** In-lens typography uses a dark semi-translucent glass plate (`rgba(6, 9, 12, 0.88)`) combined with an outer `0.1` thickness pure black outline to ensure legibility against bright sunlight or direct industrial workshop lamps.
- **Auto-Decay Minimization:** Notification cards automatically minimize after 12 seconds of inactivity to eliminate cognitive fatigue and maintain spatial awareness.

4.1 Meta Quest Platform Architecture

Meta Quest 3 and Quest Pro run **Horizon OS** (Android 12/14 base on Snapdragon XR2 Gen 2) with full 6DoF inside-out tracking, dual high-resolution RGB color passthrough cameras, and OpenXR compliance.

Quest Architectural Divergence from Spectacles

Unlike Spectacles (optical see-through waveguide with a fixed forward camera), Quest is a **video passthrough (VST) VR headset**. The cameras see the world first, composite graphics, and display them on micro-OLED/LCD panels.

4.2 Camera Passthrough & Video Streaming Pipeline

On Meta Horizon OS, applications historically could not access raw camera passthrough pixels due to privacy sandbox rules. With **Meta Quest Passthrough Camera API (v66+)** or enterprise developer entitlement:

- **Accessing Frames:** In Unity, obtain the camera feed using `PassthroughCameraAccess` or render a dedicated virtual camera capturing the technician's POV.
- **Hardware Video Encoding:** Rather than CPU-heavy JPEG compression, use Android's native `MediaCodec` (via Unity NDK or C# plugin) to encode H.264 baseline slices, reducing latency to <25ms.

```
// QuestRelayViewClient.cs - Meta Quest Passthrough Streamer (Unity C#)
using System;
using System.Net.WebSockets;
using System.Threading;
using System.Threading.Tasks;
using UnityEngine;

public class QuestRelayViewStreamer : MonoBehaviour {
    [SerializeField] private Camera povCamera;
    [SerializeField] private RenderTexture renderTarget;
    private ClientWebSocket ws = new ClientWebSocket();
    private Texture2D exportTexture;

    async void Start() {
        exportTexture = new Texture2D(renderTarget.width, renderTarget.height, TextureFormat.RGB24, false);
        await ws.ConnectAsync(new Uri("wss://relayview.allthingskrazzy.com"), CancellationToken.None);
        StartCoroutine(StreamLoop());
    }

    private System.Collections.IEnumerator StreamLoop() {
        while (ws.State == WebSocketState.Open) {
            yield return new WaitForEndOfFrame();
            RenderTexture.active = renderTarget;
            exportTexture.ReadPixels(new Rect(0, 0, renderTarget.width, renderTarget.height), 0, 0);
            exportTexture.Apply();

            byte[] jpgBytes = exportTexture.EncodeToJPG(65);
            string base64 = Convert.ToBase64String(jpgBytes) + "|||FRAME_END|||";
            byte[] sendBuffer = System.Text.Encoding.UTF8.GetBytes(base64);
            _ = ws.SendAsync(new ArraySegment<byte>(sendBuffer), WebSocketMessageType.Text, true, CancellationToken.None);
            yield return new WaitForSeconds(0.066f); // 15 FPS
        }
    }
}
```

4.3 Spatial Hit-Testing & World Anchoring on Horizon OS

Replace Lens Studio's `InstantWorldHitTest` with Meta's **Scene Mesh Raycasting** or OpenXR Spatial Entities:

```
// Quest Raycast & Anchor Implementation (Unity C#)
public void SpawnAnnotationFromWeb(float xNorm, float yNorm, string label) {
    // Convert normalized web coords to Quest Camera Viewport
    Vector3 viewportPoint = new Vector3(xNorm, yNorm, 0f);
    Ray ray = povCamera.ViewportPointToRay(viewportPoint);

    // Raycast against Meta Scene Mesh / Spatial Environment Layer
    if (Physics.Raycast(ray, out RaycastHit hit, 5.0f, LayerMask.GetMask("SpatialMesh"))) {
        GameObject pin = Instantiate(annotationPrefab, hit.point, Quaternion.LookRotation(hit.normal));
        pin.GetComponentInChildren<TMPPro.TextMeshPro>().text = label;
        // Bind to Meta OVRSpatialAnchor for persistence across headset drift
    }
}
```

```
        pin.AddComponent<OVRSpatialAnchor>();  
    }  
}
```

5.1 Android XR Architectural Overview

Android XR (built on Android 14+ / OpenXR with Jetpack XR extensions) powers the next wave of spatial headsets (Samsung XR, Google ecosystem). It combines the traditional Android application sandbox with first-class spatial panel ergonomics and ARCore Perception.

Subsystem	Snap Spectacles Baseline	Android XR Implementation
Spatial Perception	<code>InstantWorldHitTest</code>	<code>androidx.xr.scenecore.SpatialSession.hitTest()</code>
Camera Ingestion	<code>CameraModule</code>	Camera2 / CameraX Passthrough Stream API
Audio Hardware	<code>MicrophoneAudioProvider</code>	<code>AudioRecord</code> with <code>VOICE_COMMUNICATION</code> attribute
HUD & UI	<code>ScreenTransform</code>	Jetpack XR <code>SpatialPanel</code> with Compose XR

5.2 Android XR Implementation (Kotlin / Jetpack XR)

The following Kotlin component demonstrates establishing the RelayView session and binding the incoming 2D click coordinates to an Android XR `SpatialEntity`:

```
// AndroidXRRelayViewManager.kt - Android XR Jetpack Implementation
package com.allthingskrazzy.relayview

import android.content.Context
import androidx.xr.scenecore.SpatialSession
import androidx.xr.scenecore.Entity
import androidx.xr.scenecore.Transform
import okhttp3.*
import org.json.JSONObject

class AndroidXRRelayViewManager(
    private val context: Context,
    private val spatialSession: SpatialSession
) {
    private val client = OkHttpClient()
    private var websocket: WebSocket? = null

    fun connect() {
        val request = Request.Builder().url("wss://relayview.allthingskrazzy.com").build()
        websocket = client.newWebSocket(request, object : WebSocketListener() {
            override fun onOpen(ws: WebSocket, response: Response) {
                // Request ephemeral session code
                ws.send("{\"action\":\"create-session\"}")
            }

            override fun onMessage(ws: WebSocket, text: String) {
                val json = JSONObject(text)
                when (json.optString("action")) {
                    "annotate" -> handleAnnotation(json.getJSONObject("data"))
                }
            }
        })
    }

    private fun handleAnnotation(data: JSONObject) {
        val x = data.getDouble("x").toFloat()
        val y = data.getDouble("y").toFloat()
        val label = data.getString("label")

        // Android XR Perception Screen-to-World Hit Test
        val hitResult = spatialSession.hitTest(x, 1.0f - y)
        val pose = hitResult?.firstOrNull()?.pose ?: return

        // Instantiate anchored 3D spatial entity in Jetpack XR
        val pinEntity = Entity.create(spatialSession).apply {
            setTransform(Transform(pose.translation, pose.rotation))
            addComponent(SpatialTextComponent(label, color = 0xFFD4F36F.toInt()))
            addComponent(BillboardComponent()) // Always face user gaze
        }
    }
}
```


6.1 Cross-Platform Implementation Rosetta Stone

When engineering cross-platform ports, use this direct mapping between runtime frameworks:

Capability	Snap Spectacles (Lens Studio)	Meta Quest (Horizon OS / Unity)	Android XR (Jetpack XR)
WebSocket Engine	<code>internetModule.createWebSocket()</code>	<code>System.Net.WebSockets.ClientWebSocket</code>	<code>OkHttp WebSocketListener</code>
Audio Capture	<code>MicrophoneAudioProvider</code>	<code>Microphone.Start()</code> / Oboe C++	<code>AudioRecord(VOICE_COMMUNICATION)</code>
Audio Output	<code>AudioOutputProvider.enqueueAudioFrame()</code>	<code>OnAudioFilterRead()</code> / <code>AudioSource</code>	<code>AudioTrack.write(MODE_STREAM)</code>
Speech-to-Text	<code>VoiceMLModule.AsrModule</code>	Meta Wit.ai Voice SDK	Android <code>SpeechRecognizer</code>
World Raycast	<code>InstantWorldHitTest.hitTest()</code>	<code>Physics.Raycast(SpatialMeshLayer)</code>	<code>SpatialSession.hitTest()</code>
HUD Display	<code>Component.ScreenTransform</code>	World Space Canvas (Lazy Follow)	<code>SpatialPanel</code> (Compose XR)

6.2 Production Scaling & Infrastructure Roadmap

RelayView's backend is currently deployed as a stateless serverless container on **Google Cloud Run** in `us-central1` behind custom domain routing (`relayview.allthingskrazyy.com`). To scale beyond single-instance concurrent limits:

- **Horizontal Scale with Redis Pub/Sub:** Replace in-memory `Map<sessionCode, session>` with Redis. When a web client connects to Instance A and a technician to Instance B, audio and video packets are published to a Redis channel keyed by `session:{code}`.
- **Hardware H.264 WebRTC Upgrade:** For high-bandwidth 60 FPS streaming on Quest and Android XR, implement WebRTC data channels with DTLS/SRTP encryption, keeping the WebSocket relay as signaling.
- **Spatial Audio Spatialization:** On 6DoF platforms (Quest / Android XR), position the remote expert's voice in 3D audio space directly at the coordinate of their placed annotation, creating intuitive auditory localization.

✓ Deployment Verification & Live Status

The reference production server is deployed and verified active:

Endpoint: <https://relayview.allthingskrazyy.com>

Open-Source Repository: <https://github.com/kgediya/Remote-ARsistance>